INITIAL DEFENCE PRESENTATION

COMPUTER SCIENCE DEPARTMENT

Quantum-Inspired Network Intrusion Detection System

Optimizing ML-based NIDS using Quantum-Inspired Algorithms for Feature Selection & Classification

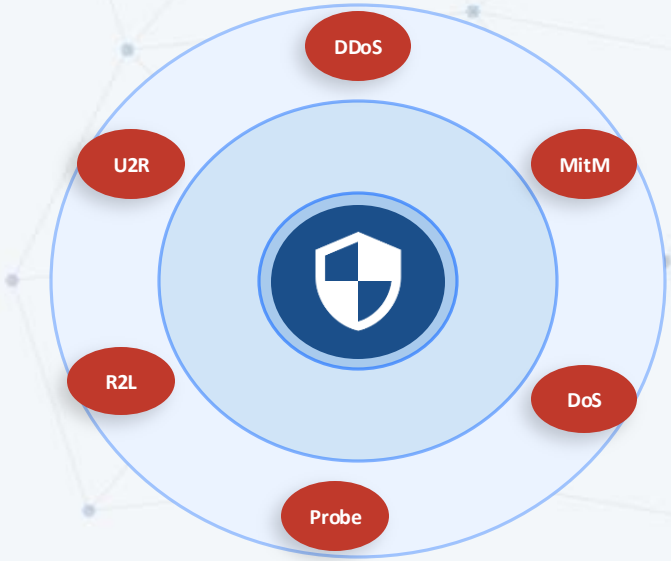
Presented by	Roll No	Supervisor	Session
Anita Bashir	**92**	**Prof. Dr Syed Ali Abbas**	**2022–2026**
Nayab Bashir	**74**		



PROBLEM STATEMENT

Why Do We Need This System?

Network intrusion is a growing threat — existing solutions fall short



Global

- $10.5 Trillion cybercrime cost by 2025
- Cyberattacks ↑ 15% every year
- Traditional NIDS = high false-positive rates

PK Nationally

- Banks & telecom sectors under attack
- No indigenous cybersecurity tools
- Real-time detection gap in critical infra

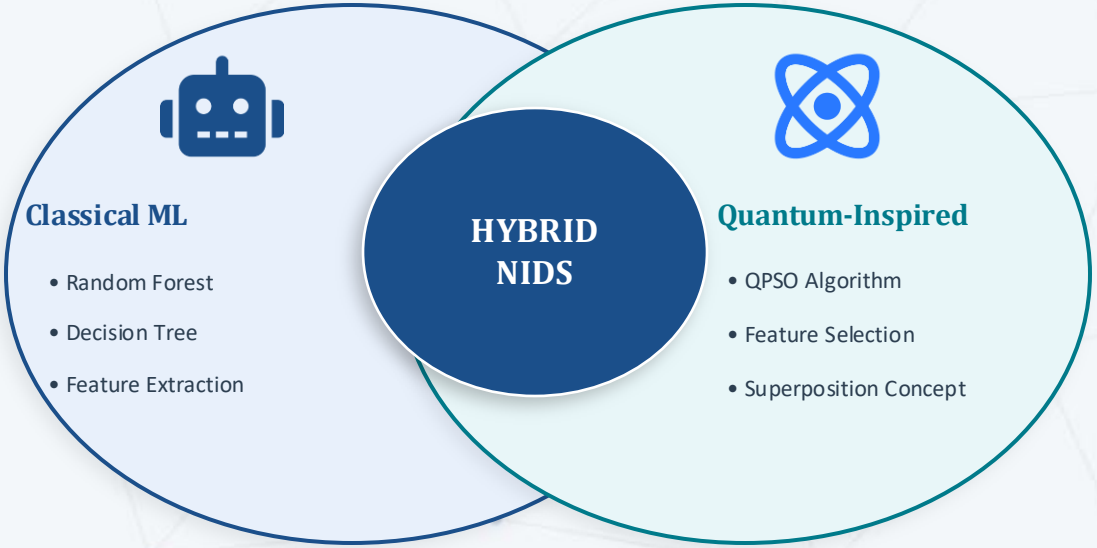
Local

- Campus networks unprotected
- Manual monitoring — inefficient
- Research gap: quantum-ML NIDS not explored

PROPOSED SOLUTION

Hybrid ML-NIDS with Quantum-Inspired Optimization

Combining classical ML with QPSO for smarter, faster intrusion detection



Smart Feature Selection

41 features → 18-22 via QPSO

Higher Accuracy

~88% → ~94%+ detection rate

Lower False Positives

12% FPR → ~5% FPR

GUI Simulator

Visual dashboard + report

Phase 1 & 2 — Data Preprocessing & Classical ML

① Data Prep & Classical ML | ② QPSO Optimization | ③ Simulation & GUI

Phase 1

Dataset
NSL-KDD / UNSW-NB15

→

Clean
Remove nulls & dupes

→

Normalize
Min-Max / Z-Score

→

80/20 Split
Train & Test sets

Phase 2

Preprocessed Data
Ready for training

→

Train RF & DT Models
Random Forest + Dec. Tree

→

Evaluate Results
Acc · Prec · Recall · FPR

→

Baseline Saved
Reference for Phase 2

Phase 3 — QPSO Optimization

1. Classical Baseline Results → Input to QPSO
2. Initialize Particles (each = feature subset)
3. Evaluate Fitness (classification accuracy per subset)
4. Update Quantum Velocity & Position
5. Converge → Select Optimal Feature Subset

iterate until converge

↓ Reduce False Positives

↑ Improve Accuracy

Faster Classification

Phase 4 — Simulation & GUI Development

① Data Prep & Classical ML | ② QPSO Optimization | ③ Simulation & GUI

Backend — Model Pipeline

- Load Trained Models (RF, DT, Hybrid)
- Accept Network Data Input
- Run QPSO Feature Selection
- Classify via Hybrid RF Model
- Generate Prediction & Confidence Score

Frontend — GUI (Python Tkinter/PyQt5)

- User Uploads / Inputs Network Traffic
- Display Live Detection Results
- Show Metrics (charts/stats)
- Classical vs Quantum Comparison View
- Custom Attack Injection

Complete Simulation System

EXPECTED RESULTS & CONCLUSION

Measurable Improvements Over Classical ML

Based on literature benchmarks for similar hybrid NIDS approaches

Detection Accuracy	False Positive Rate	Feature Count	Processing Speed
Before: ~88%	Before: ~12%	Before: 41	Before: base
After: ~94%+	After: ~5%	After: ~18–22	After: +30% faster

- Quantum-Inspired NIDS System
- NSL-KDD & UNSW-NB15
- QPSO — 41 to 19 features
- Python GUI Simulator

Novel contribution to cybersecurity research in Pakistan — achievable within a 4-month timeline

TOOLS & TECHNOLOGIES

Technical Stack

All tools are free, open-source, and well-documented

Core Language

- Python 3.x
- Jupyter Notebook
- VS Code IDE

ML Libraries

- scikit-learn
- TensorFlow / Keras
- NumPy / Pandas

Quantum (QPSO)

- DEAP library
- pyswarm
- Custom QPSO impl.

Visualization

- Matplotlib
- Seaborn
- Plotly

GUI Framework

- Tkinter / PyQt5
- Dark dashboard
- Live chart embed

Datasets

- NSL-KDD (UNB.ca)
- UNSW-NB15
- Publicly available

Expected Interface

